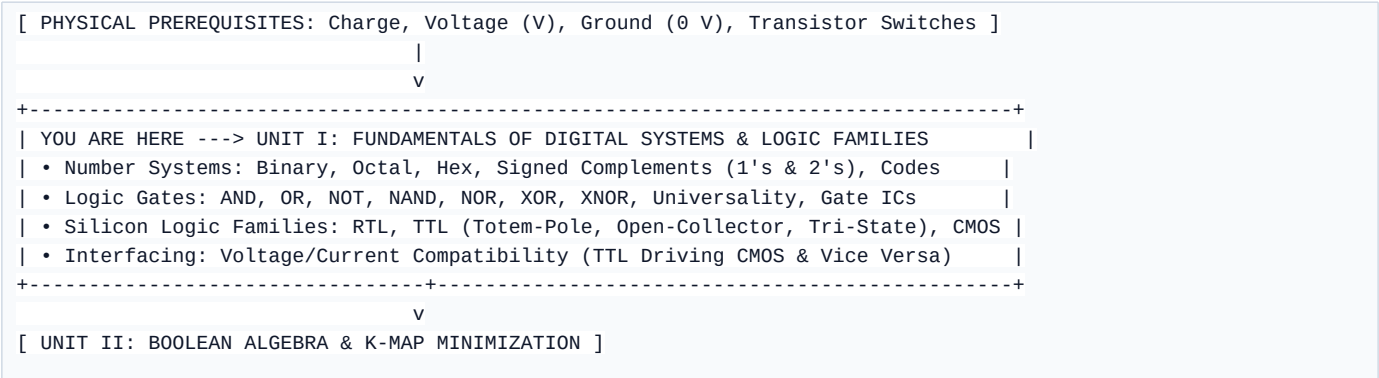


DEC Unit I: Fundamentals & Logic Families

Comprehensive First-Principles Lecture Notes

Unit I: Fundamentals of Digital Systems and Logic Families

Master Subject Map: Unit I Position



In 3 lines: Unit I establishes the language of digital electronics. We translate human numbers into patterns of binary voltages, use semiconductor transistors to construct primitive decision-making blocks called logic gates, and analyze the physical silicon families (RTL, TTL, CMOS) that power real integrated circuits.

Prerequisites for this Unit

Before studying Unit I, you need the foundational concepts established in the Prerequisites section: 1. **Voltage and Ground (Prereq 1 & 2):** Current flows from a positive potential (V_{CC} / V_{DD}) to Ground (0 V). 2. **Transistors as Electronic Switches (Prereq 3):** An active control input creates a conducting short to ground (pull-down), while absence of control leaves the path open (pull-up). 3. **Logic Levels & Voltage Bands (Prereq 4):** A HIGH signal is a voltage in the upper band (near $+5\text{ V}$), while a LOW signal is in the lower band (near 0 V). 4. **Positional Numbering & Powers of Two (Prereq 6):** Powers of two (**1, 2, 4, 8, 16 dots**) form the basis of binary representations.

PART 1: Number Systems, Signed Binary Arithmetic, and Codes

1.1 Number Base Systems: Binary, Octal, and Hexadecimal

Source: Kumar (Ch. 2, pp. 59–68); Floyd (Ch. 2, pp. 52–65); Mano (Ch. 1, pp. 18–28)

[Pacing: Systematic Bookkeeping — Follow the Step-by-Step Algorithm]

Need (Problem-First)

Human beings communicate in decimal (base 10), but computer processors physically store and manipulate bits in base 2. However, long strings of binary digits (such as **1101111010101101_2**) are notoriously difficult for human engineers to read, write, and debug without making transcription errors. We need compact, human-friendly representations that map directly into binary bits without requiring painful mathematical long-division. This creates the need for **Octal (base 8)** and **Hexadecimal (base 16)**.

Chain of Cause and Effect

Binary is machine-native but unwieldy for humans → bases that are exact powers of two ($8 = 2^3$ and $16 = 2^4$) allow direct bit-grouping → each octal digit corresponds to exactly 3 binary bits, and each hexadecimal digit corresponds to exactly 4 binary bits → complex memory addresses and machine instructions can be inspected with ease.

How It Works (Mechanism & Algorithms)

1. Positional Bases Defined

- **Decimal (Base 10):** Radix $r=10$. Digits in $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.
- **Binary (Base 2):** Radix $r=2$. Digits in $\{0, 1\}$.
- **Octal (Base 8):** Radix $r=8$. Digits in $\{0, 1, 2, 3, 4, 5, 6, 7\}$.
- **Hexadecimal (Base 16):** Radix $r=16$. Digits in $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$.

Where letters represent numbers 10 through 15 inline:

$$A_{\{16\}} = 10_{\{10\}}, B_{\{16\}} = 11_{\{10\}}, C_{\{16\}} = 12_{\{10\}}, D_{\{16\}} = 13_{\{10\}}, E_{\{16\}} = 14_{\{10\}}, F_{\{16\}} = 15_{\{10\}}$$

2. Conversions to Decimal (Sum-of-Weights Method)

Multiply each digit by its positional weight r^i and compute the arithmetic sum (Kumar, p. 63). - **Binary to Decimal:**

Example: Convert 11010.11_2 to decimal:

$$11010.11_2 = (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) + (1 \times 2^{-1}) + (1 \times 2^{-2})$$

$$= 16 + 8 + 0 + 2 + 0 + 0.5 + 0.25 = 26.75_{\{10\}}$$

- **Hexadecimal to Decimal:**

Example: Convert $2A6_{\{16\}}$ to decimal:

$$2A6_{\{16\}} = (2 \times 16^2) + (10 \times 16^1) + (6 \times 16^0) = (2 \times 256) + (10 \times 16) + (6 \times 1) = 512 + 160 + 6 = 678_{\{10\}}$$

3. Decimal to Binary Conversion (Double-Dabble Method)

- **Integer Part (Repeated Division by 2):** Divide the decimal integer repeatedly by 2, recording the remainder (0 or 1) at each step, until the quotient reaches 0. The **first remainder is the LSB**, and the **last remainder is the MSB** (Kumar, p. 64).

Example: Convert $25_{\{10\}}$ to binary:

- $25 \div 2 = 12$, remainder 1 (LSB)
- $12 \div 2 = 6$, remainder 0
- $6 \div 2 = 3$, remainder 0
- $3 \div 2 = 1$, remainder 1
- $1 \div 2 = 0$, remainder 1 (MSB)

Reading remainders from bottom to top: $25_{\{10\}} = 11001_2$.

- **Fractional Part (Repeated Multiplication by 2):** Multiply the fraction repeatedly by 2, recording the integer carry (0 or 1) generated at each step, until the fraction becomes zero or the desired precision is reached. The **first integer carry is the first fractional digit** (Kumar, p. 66). Example: Convert $0.625_{\{10\}}$ to binary:

- $0.625 \times 2 = 1.250$ (carry 1)
- $0.250 \times 2 = 0.500$ (carry 0)
- $0.500 \times 2 = 1.000$ (carry 1, fraction zeroed out)

Reading carries top to bottom: $0.625_{\{10\}} = 0.101_2$.

4. Octal and Hexadecimal Bit-Grouping (The Fast Direct Shortcut)

Because $8 = 2^3$, every octal digit converts into exactly 3 binary bits.

Because $16 = 2^4$, every hexadecimal digit converts into exactly 4 binary bits. - **Binary to Hexadecimal:** Group bits into sets of 4 starting from the binary point moving left for integers and right for fractions (padding with leading/trailing zeros if needed), then substitute the hex digit: Example: Convert 1011110010_2 to hex: - Group: $\underbrace{0010}_{\{2\}} \underbrace{1111}_{\{F\}} \underbrace{0010}_{\{2\}}$ (padded with two leading zeros on MSB group) - Result: $1011110010_2 =$

2F2{16}. - Hexadecimal to Binary: Directly expand each hex digit into its 4-bit binary equivalent: *Example:* Convert **3B_{16}** to binary: - **3_{16} = 0011_2** - **B_{16} = 11_{10} = 1011_2** - Result: **3B_{16} = 00111011_2**.

Understanding Checkpoint

- **Question:** Convert **11110110_2** directly to hexadecimal.
- **Answer:** Split into two 4-bit groups: **1111_2** and **0110_2**. **1111_2 = 15_{10} = F_{16}**, and **0110_2 = 6_{16}**. Therefore, **11110110_2 = F6_{16}**.

1.2 Binary Arithmetic: Addition, Subtraction, Multiplication, and Division

Source: Kumar (Ch. 2, pp. 68–71); Floyd (Ch. 2, pp. 58–63)

[Pacing: Routine Bookkeeping — Master the Elementary Rules]

Need (Problem-First)

To make hardware compute, we must translate mathematical operations into primitive bit manipulations that can be executed by electronic logic circuits.

The Four Elementary Rules of Binary Arithmetic

1. **Binary Addition (Kumar, p. 68):**
2. **0 + 0 = 0** (Sum 0, Carry 0)
3. **0 + 1 = 1** (Sum 1, Carry 0)
4. **1 + 0 = 1** (Sum 1, Carry 0)
5. **1 + 1 = 10_2** (Sum 0, Carry 1 to the next higher positional column)
6. **1 + 1 + 1 = 11_2** (Sum 1, Carry 1) *Worked Example:* Add **1101_2 (13_{10})** and **0111_2 (7_{10})**: `` 1 1 1 <-- Carry
row 1 1 0 1 (13)
 - 0 1 1 1 (7)

1 0 1 0 0 (20 in decimal: 16 + 4 = 20) ``

7. **Binary Subtraction (Kumar, p. 68):**
8. **0 - 0 = 0**
9. **1 - 0 = 1**
10. **1 - 1 = 0**
11. **0 - 1 = 1** with a **Borrow of 1** from the next higher column (which has a weight of 2).
12. **Binary Multiplication (Kumar, p. 69):** Identical to decimal long multiplication, but vastly simpler because multiplier bits are only 0 or 1. A 1 copies the multiplicand; a 0 yields all zeros.
13. **Binary Division (Kumar, p. 70):** Performed by successive shifting and subtraction, matching long division in decimal.

Understanding Checkpoint

- **Question:** Compute **101_2 + 011_2**.
- **Answer:** **101_2 (5) + 011_2 (3) = 1000_2 (8)**.

1.3 Signed Binary Numbers: Sign-Magnitude, 1's Complement, and 2's Complement

Source: Kumar (Ch. 2, pp. 71–85); Mano (Ch. 1, pp. 34–39); Floyd (Ch. 2, pp. 65–75)

[Pacing: Conceptually Crucial — Read Slowly and Methodically]

Need (Problem-First)

In everyday math, we indicate negative numbers by simply writing a minus sign "-" in front (e.g., **-15**). But an electronic circuit wire can only carry two physical voltage states: HIGH or LOW. It cannot carry a "-" symbol. How can hardware represent negative numbers, and more importantly, how can an Arithmetic Logic Unit (ALU) perform subtraction *without having to build completely separate, expensive subtraction circuitry*?

Chain of Cause and Effect

Circuits allocate the leftmost bit (MSB) as a sign indicator (**0 = +**, **1 = -**) → direct sign-magnitude creates two zeros (**+0** and **-0**) and requires complex decision logic → complement systems convert subtraction into simple addition (**A - B = A + (-B)**) → 2's complement eliminates duplicate zeros and discards the final carry → the exact same hardware adder circuit performs

both addition and subtraction.

How It Works (The Three Signed Formats Defined)

In an n -bit binary word, the **Most Significant Bit (MSB)** is strictly reserved as the **Sign Bit**: - **MSB = 0** ■ **Positive number (+)** - **MSB = 1** ■ **Negative number (-)**

1. Sign-Magnitude Representation (Kumar, p. 71)

The MSB indicates the sign, and the remaining $(n-1)$ bits represent the absolute magnitude of the number: - For an 8-bit byte: - **$+25_{10} = \text{mathbf{0}0011001_2$** - **$-25_{10} = \text{mathbf{1}0011001_2$** *Fatal Engineering Flaw*: Sign-magnitude has **two distinct representations for zero**: **$+0 = 00000000_2$** and **$-0 = 10000000_2$** . This wastes bit combinations and forces processors to execute extra comparison cycles.

2. 1's Complement Representation (Kumar, p. 72)

- Positive numbers are written in standard binary with **MSB = 0**.
- A negative number is formed by taking the positive binary number and **inverting every single bit** (**0 $\rightarrow$ 1** and **1 $\rightarrow$ 0**).
- *Example (8 bits): $+25_{10} = 00011001_2$.*
To form **-25_{10}** , invert all bits: **$-25_{10} = 11100110_2$** . *Limitation*: Like sign-magnitude, 1's complement still suffers from two zeros (**$+0 = 00000000_2$** and **$-0 = 11111111_2$**). Furthermore, subtraction produces an "end-around carry" that must be re-added to the LSB, slowing down the processor.

3. 2's Complement Representation (Kumar, p. 74)

The universal standard for all modern computing architectures. - Positive numbers are written in standard binary with **MSB = 0**. - A negative number is formed by taking its 1's complement and **adding 1 to the LSB**:

$$\text{2's Complement} = (\text{1's Complement}) + 1$$

- *Worked Example (8-bit representation of -25_{10}):*
- Start with positive **$+25_{10}$** : 00011001
- Invert all bits (1's complement): 11100110
- Add 1: $\begin{array}{r} 11100110 \\ + 1 \\ \hline \end{array}$

$11100111 \leftarrow -25$ in 2's complement $\begin{array}{r} 11100110 \\ + 1 \\ \hline \end{array}$

- *The Direct Shortcut for 2's Complement*: Working from right to left (LSB to MSB), copy all bits unchanged up to and including the first 1, then invert every bit remaining to the left!
For 00011001 : first bit is 1 (copy it), invert rest $\rightarrow 11100111$. Exactly identical result in one second!

4. Subtraction Using 2's Complement Arithmetic (Kumar, p. 81)

To compute **$A - B$** , the ALU converts the problem into:

$$A - B = A + (\text{2's complement of } B)$$

The Two Golden Rules of 2's Complement Subtraction: 1. If a final carry of 1 is generated past the MSB, **DISCARD IT**. The remaining bits are the correct positive answer in standard binary (Kumar, p. 81). 2. If **NO final carry is generated, the result is NEGATIVE and is in 2's complement form**. To read its human-readable decimal magnitude, take the 2's complement of the result and attach a minus sign (Kumar, p. 81).

Concrete Worked Scenario 1 (Larger minus Smaller: $28 - 15$ in 8 bits): - **$+28_{10} = 00011100_2$** - **$+15_{10} = 00001111_2$** ■ **-15_{10}** in 2's complement = **$11110000 + 1 = 11110001_2$** . - Add them: $\begin{array}{r} 00011100 \text{ (+28)} \\ + 11110001 \text{ (-15)} \\ \hline \end{array}$

$1\ 00001101$ ^ Discard this end carry! $\begin{array}{r} 00001101 \\ \hline \end{array}$ Result = **$00001101_2 = 8 + 4 + 1 = +13_{10}$** . Perfect!

Concrete Worked Scenario 2 (Smaller minus Larger: $15 - 28$ in 8 bits): - **$+15_{10} = 00001111_2$** - **$-28_{10} = 11110001_2$** (2's complement of 28) - Add them: $\begin{array}{r} 00001111 \text{ (+15)} \\ + 11110001 \text{ (-28)} \\ \hline \end{array}$

11110011 (No end carry generated!)

Since no carry appeared, the result is negative and stored in 2's complement. To find its magnitude: take 2's complement of 11110011 $\rightarrow$ invert bits (00001100) + 1 = 00001101 (**13**₁₀).

Therefore, the answer is **-13**₁₀. Perfect!

5. Arithmetic Overflow

Source: Kumar (p. 84); Mano (p. 37)

In an n -bit signed system, the range of representable numbers is:

$$-[2^{n-1}] \text{ to } +[2^{n-1} - 1]$$

For an 8-bit system: **-128** to **+127**.

Overflow occurs if the sum of two numbers with the same sign exceeds this range.

- If you add two positive numbers and the MSB becomes 1 (negative), **overflow has occurred**. - If you add two negative numbers and the MSB becomes 0 (positive), **overflow has occurred**. - Hardware detection rule: Overflow occurs if and only if the **carry into the sign bit (C_{in})** is different from the **carry out of the sign bit (C_{out})**:

$$\text{Overflow} = C_{in} \oplus C_{out}$$

Understanding Checkpoint

- **Question:** What is the 8-bit 2's complement representation of **-1**?
- **Answer:** **+1 = 00000001**₂. Invert all bits $\rightarrow$ **11111110**₂. Add **1** $\rightarrow$ **11111111**₂. Thus, **-1** is represented as all ones (11111111).

1.4 Binary Codes: BCD, Excess-3, Gray Code, and ASCII

Source: Kumar (Ch. 3, pp. 117–134); Floyd (Ch. 2, pp. 75–85); Mano (Ch. 1, pp. 39–48)

[Pacing: Systematic Vocabulary — High Relevance to Practical Displays]

Need (Problem-First)

Raw binary arithmetic is great for calculations, but digital systems must interface with decimal keypads, seven-segment displays, mechanical rotary encoders, and human text. If you directly send pure binary to a 7-segment display driver, the circuitry required to decode 16-bit numbers into decimal digits becomes massive. Specialized **codes** bridge these real-world interfaces.

The Four Core Codes Defined

1. Binary Coded Decimal (BCD / 8421 Code)

Source: Kumar (Ch. 3, pp. 117–120)

- **Mechanism:** Each decimal digit (0 through 9) is independently encoded using its 4-bit binary equivalent with weights **8, 4, 2, 1**. - Because 4 bits can represent 16 states (0 to 15), the combinations from **10**₁₀ to **15**₁₀ (1010 to 1111) are **strictly invalid in BCD**. - Example: Convert **952**₁₀ to BCD:

$$9_{10} = 1001_2, 5_{10} = 0101_2, 2_{10} = 0010_2 \blacksquare 952_{\text{BCD}} = 1001 \ 0101 \ 0010$$

Notice that **952**_{BCD} requires 12 bits, whereas pure binary **952**₁₀ requires only 10 bits (**1110111000**₂). BCD trades storage density for effortless decimal interfacing.

2. Excess-3 Code (XS-3)

Source: Kumar (Ch. 3, pp. 120–122)

- **Mechanism:** An unweighted code derived by adding decimal **3** (**0011**₂) to each BCD code group:

$$\text{XS-3} = \text{BCD} + 0011_2$$

Example: Decimal **4** $\rightarrow$ BCD 0100 $\rightarrow$ XS-3 = 0100 + 0011 = 0111. - **Why It Matters (Self-Complementing Property):** The 1's complement of an Excess-3 number produces the Excess-3 code of its 9's complement! This made XS-3 immensely valuable in

early computing hardware for automating decimal subtraction.

3. Gray Code (Reflected Binary / Unit Distance Code)

Source: Kumar (Ch. 3, pp. 122–126); Floyd (Ch. 2, pp. 78–81)

- **Need:** When an optical rotary shaft encoder measures the position of a rotating motor, standard binary transitions can be catastrophic. When transitioning from state **3** (011) to state **4** (100), **all three bits must change simultaneously**. Because physical sensor contacts never switch in perfect mathematical synchrony, the encoder may momentarily register spurious intermediate states like 000 or 111, causing false position errors. - **The Gray Code Solution:** In Gray code, **only one bit changes state at a time** between any two consecutive numbers (Unit Distance property). - **Binary to Gray Code Conversion (Kumar, p. 123):** 1. The MSB of the Gray code is identical to the MSB of the binary number: $G_n = B_n$. 2. Each subsequent Gray bit G_i is formed by XORing the current binary bit B_{i+1} with the next lower binary bit B_i :

$$G_i = B_{i+1} \oplus B_i$$

Worked Example: Convert binary **1101₂** to Gray code: - $G_3 = B_3 = 1$ - $G_2 = B_3 \oplus B_2 = 1 \oplus 1 = 0$ - $G_1 = B_2 \oplus B_1 = 1 \oplus 0 = 1$ - $G_0 = B_1 \oplus B_0 = 0 \oplus 1 = 1$

Result: **1101₂ = 1011_{Gray}**. - **Gray Code to Binary Conversion (Kumar, p. 124):** 1. The MSB of the binary number is identical to the Gray MSB: $B_n = G_n$. 2. Each subsequent binary bit is obtained by XORing the previously calculated binary bit with the current Gray bit:

$$B_i = B_{i+1} \oplus G_i$$

4. Alphanumeric Code: ASCII

Source: Floyd (Ch. 2, pp. 83–85); Mano (Ch. 1, pp. 44–46)

The **American Standard Code for Information Interchange (ASCII)** uses 7 bits to encode 128 characters, including upper and lowercase letters, numbers, punctuation, and control characters (e.g., 'A' = **1000001₂ = 41_{16} = 65_{10}**; '0' = **0110000₂ = 30_{16} = 48_{10}**).

Understanding Checkpoint

- **Question:** What is the Gray code for binary **1000₂**?
- **Answer:** $G_3 = 1$
 $G_2 = 1 \oplus 0 = 1$
 $G_1 = 0 \oplus 0 = 0$
 $G_0 = 0 \oplus 0 = 0$
Gray code = **1100**.

PART 2: Basic Digital Circuits, Universal Gates, and Gate ICs

1.5 The Fundamental Logic Gates: AND, OR, and NOT

Source: Kumar (Ch. 4, pp. 163–168); Mano (Ch. 2, pp. 84–90); Floyd (Ch. 3, pp. 118–138)

[Pacing: Core Visual Foundation — Study the Truth Tables & Symbols]

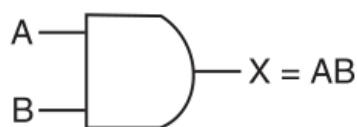
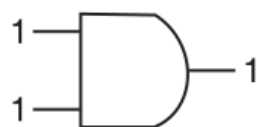
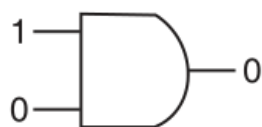
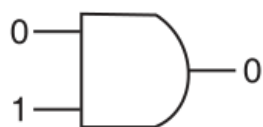
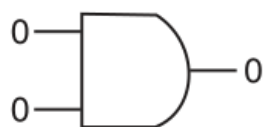
Need (Problem-First)

Mathematical statements require logical operators: "Activate the emergency cooling pump if the temperature is HIGH **AND** pressure is HIGH." "Sound the alarm if door 1 is open **OR** door 2 is open." We need hardware circuits that execute these logical decisions.

The Three Fundamental Gates

1. The AND Gate (Kumar, p. 164)

- **Plain Words:** The output is HIGH (**1**) if and only if all inputs are HIGH (**1**). If any input is LOW (**0**), the output is clamped to LOW (**0**).
- **Boolean Equation:** $X = A \cdot B$ (or simply $X = AB$).
- **Authentic Diagram:**



(a) Logic symbol

Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

(b) Truth table

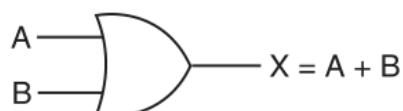
Figure 4.1 A two-input AND gate.

Figure

4.1: A two-input AND gate showing operational logic states, circuit symbol, and complete truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 164 (printed p. 132).

2. The OR Gate (Kumar, p. 166)

- **Plain Words:** The output is HIGH (1) if any input is HIGH (1). The output is LOW (0) only when all inputs are simultaneously LOW (0).
- **Boolean Equation:** $X = A + B$.
- **Authentic Diagram:**



(a) Logic symbol

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

(b) Truth table

Figure 4.4 A two-input OR gate.

Figure

4.4: A two-input OR gate showing operational logic states, circuit symbol, and complete truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 166 (printed p. 134).

3. The NOT Gate (Inverter) (Kumar, p. 168)

- **Plain Words:** The output is the exact logical complement (inverse) of the input. If the input is 0, the output is 1; if the input is 1, the output is 0.
- **Boolean Equation:** $X = A$ (or $X = A'$).
- **The Inversion Bubble:** In schematic convention, the triangle represents a buffer (current amplifier), and the small circular bubble represents logical inversion.
- **Authentic Diagram:**

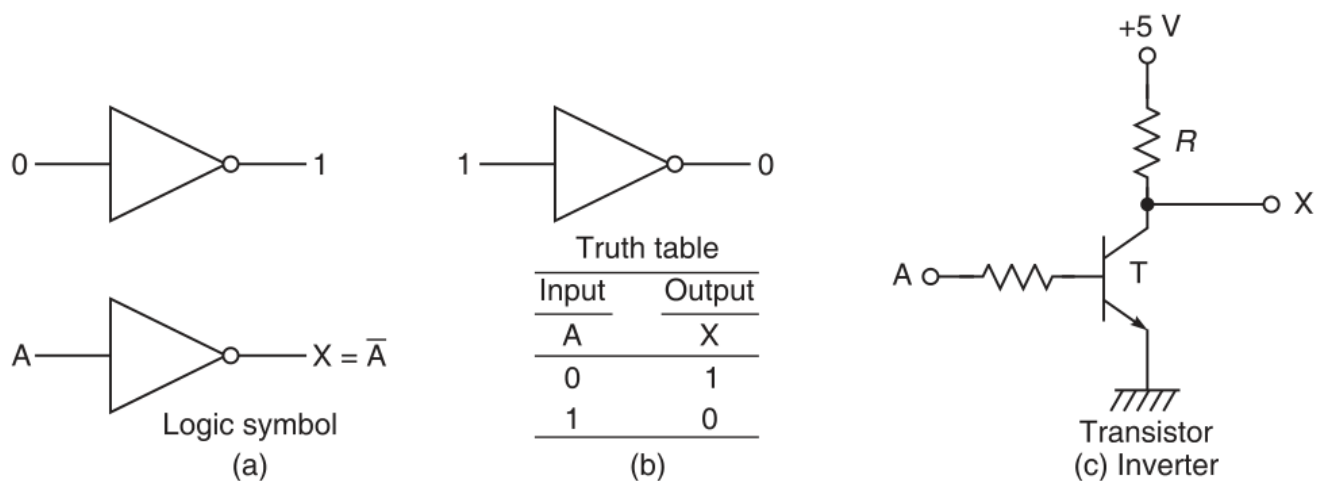


Figure 4.7 The inverter.

Figure

4.7: The logic symbol and truth table of an inverter (NOT gate). Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 168 (printed p. 136).

1.6 Universal Gates: NAND and NOR Operations & Proofs of Universality

Source: Kumar (Ch. 4, pp. 169–173); Mano (Ch. 3, pp. 119–126)

[Pacing: Conceptually Crucial — Read Carefully]

Need (Problem-First)

In manufacturing semiconductor integrated circuits, fabricating multiple distinct types of gates (some AND, some OR, some NOT) requires complex, multi-step silicon doping masks, driving up production costs and inventory complexity. Semiconductor manufacturers sought a **single universal building block** that could be mass-produced identically and wired to perform any Boolean function whatsoever. Those blocks are **NAND** and **NOR**.

The Universal Gates Defined

1. The NAND Gate (Kumar, p. 169)

- **Plain Words:** A combination of an AND gate followed immediately by an inverter. The output is LOW (0) **only when all inputs are HIGH (1)**. For all other combinations, the output is HIGH (1).
- **Boolean Equation:** $X = A \cdot B$.
- **Authentic Diagram:**

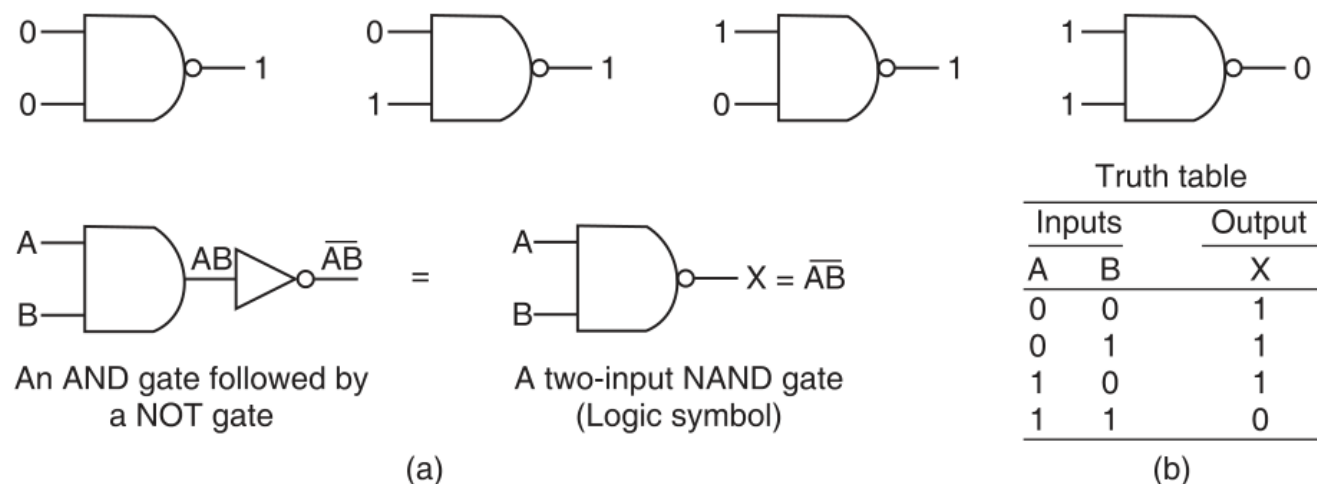


Figure 4.8 A two-input NAND gate.

Figure

4.8: A two-input NAND gate showing operational states, logic symbol with inversion bubble, and truth table. Sourced from A. Anand Kumar, *Fundamentals*

2. The NOR Gate (Kumar, p. 171)

- **Plain Words:** A combination of an OR gate followed by an inverter. The output is HIGH (1) only when all inputs are LOW (0). If any input is HIGH (1), the output is LOW (0).
- **Boolean Equation:** $X = A + B$.
- **Authentic Diagram:**

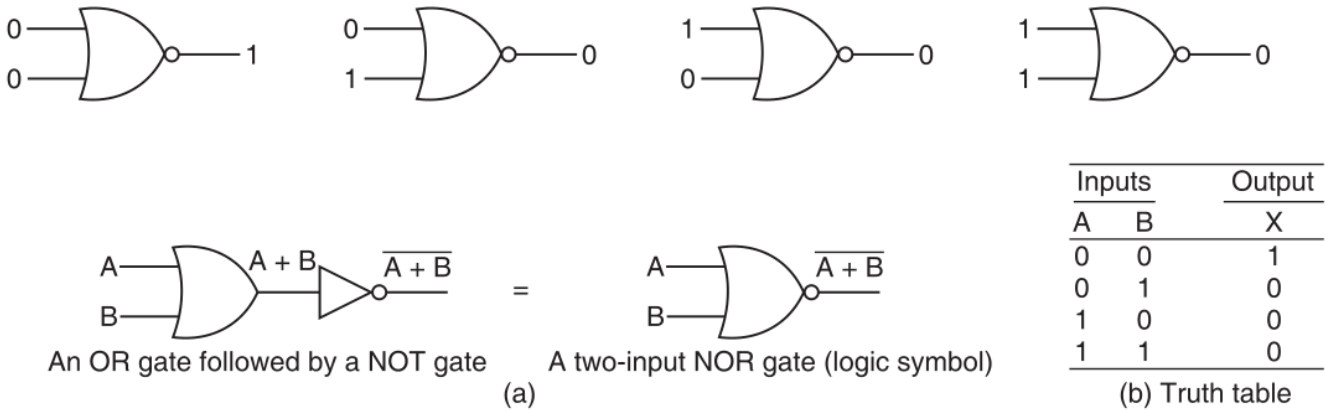


Figure 4.14 A two-input NOR gate.

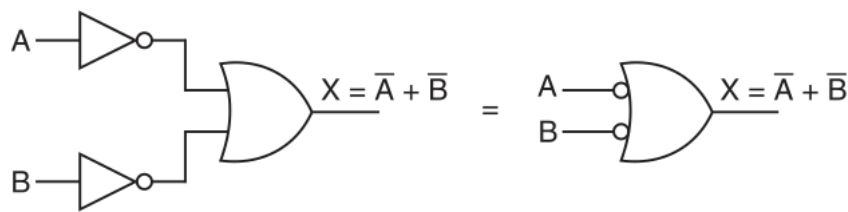
Figure

4.14: A two-input NOR gate showing operational states, logic symbol, and truth table. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 171 (printed p. 139).

Proofs of Universality: Realizing NOT, AND, and OR

1. Universality of the NAND Gate (Kumar, pp. 170–171)

- **Realization of NOT using NAND (Kumar, Figure 4.11):**
Tie both inputs of a 2-input NAND gate together: $X = A \cdot A = A$.



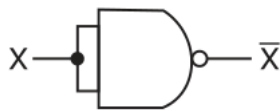
(a)

Inputs		Inverted inputs		Output
A	B	$\bar{A}$	$\bar{B}$	X
0	0	1	1	1
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

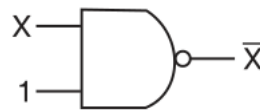
(b) Truth table

Figure 4.10 Bubbled OR gate.

NAND gate as an inverter: A NAND gate can also be used as an inverter by tying all its input terminals together and applying the signal to be inverted to the common terminal (Figure 4.11a). Alternatively, by connecting all its input terminals except one, to logic 1 and applying the signal to be inverted to the remaining terminal as shown in Figure 4.11b. In the latter form, it is said to act as a controlled inverter.



(a) NAND gate as an inverter



(b) NAND gate as a controlled inverter

Figure 4.11 NAND gate as an inverter.

Figure

4.11: Realizing a NOT gate (inverter) using a NAND gate by tying inputs together or using a controlled HIGH input. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 170 (printed p. 138).

- Realization of AND using NAND:**

Pass inputs through a NAND gate, then invert the result with a second NAND-inverter:

$$X = A \cdot B = A \cdot B$$

- Realization of OR using NAND (De Morgan's Equivalence):**

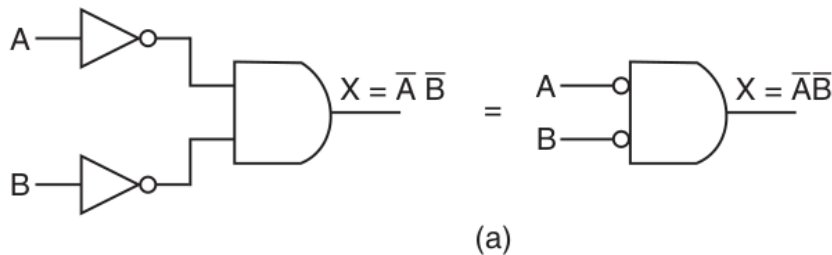
Invert **A** with one NAND to get $\bar{A}$; invert **B** with another NAND to get $\bar{B}$; feed both into a third NAND gate:

$$X = A \cdot B = A + B = A + B$$

2. Universality of the NOR Gate (Kumar, pp. 171–172)

- Realization of NOT using NOR (Kumar, Figure 4.17):**

Tie both inputs together: $X = A + A = A$.



Inputs		Inverted inputs		Output
A	B	$\bar{A}$	$\bar{B}$	X
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	0

Figure 4.16 Bubbled AND gate.

The AND gate with inverted inputs (Figure 4.16a) is called a *bubbled* AND gate. So, it is equivalent to a bubbled AND gate whose truth table is shown in Figure 4.16b. A bubbled AND gate is also called a *negative* AND gate. Since its output assumes the HIGH state only when all inputs are in LOW state, a NOR gate is also called an active-LOW AND gate.

NOR gate as an inverter: A NOR gate can also be used as an inverter, by tying all its input terminals together and applying the signal to be inverted to the common terminal (Figure 4.17a). Alternatively, connecting all its input terminals except one to logic 0, and applying the signal to be inverted to the remaining terminal as shown in Figure 4.17b. In the latter form it is said to act as a controlled inverter.

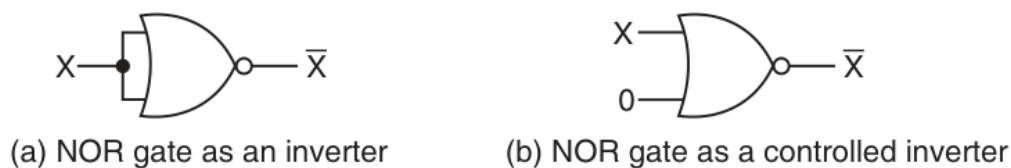


Figure 4.17 NOR gate as an inverter.

4.17: Realizing a NOT gate using a NOR gate. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 172 (printed p. 140).

- Realization of OR using NOR:**

Feed inputs to a NOR gate, then invert with a second NOR-inverter:

$$X = A + B = \overline{\overline{A + B}} = \overline{A \cdot B}$$

- Realization of AND using NOR (De Morgan's Equivalence):**

Invert inputs to get $\bar{A}$ and $\bar{B}$, then feed into a third NOR gate:

$$X = \overline{\overline{A + B}} = \overline{\overline{A} \cdot \overline{B}} = A \cdot B$$

3. Two-Level NAND-NAND Implementation (*Mano*, p. 120)

Because an AND-OR network directly converts into a two-level NAND-NAND network under De Morgan's theorem, any Sum-of-Products (SOP) expression can be wired using **only NAND gates**:

AND gates are replaced by NAND gates and the OR gate is replaced by a NAND gate with an invert-OR graphic symbol. Remember that a bubble denotes complementation and two bubbles along the same line represent double complementation, so both can be

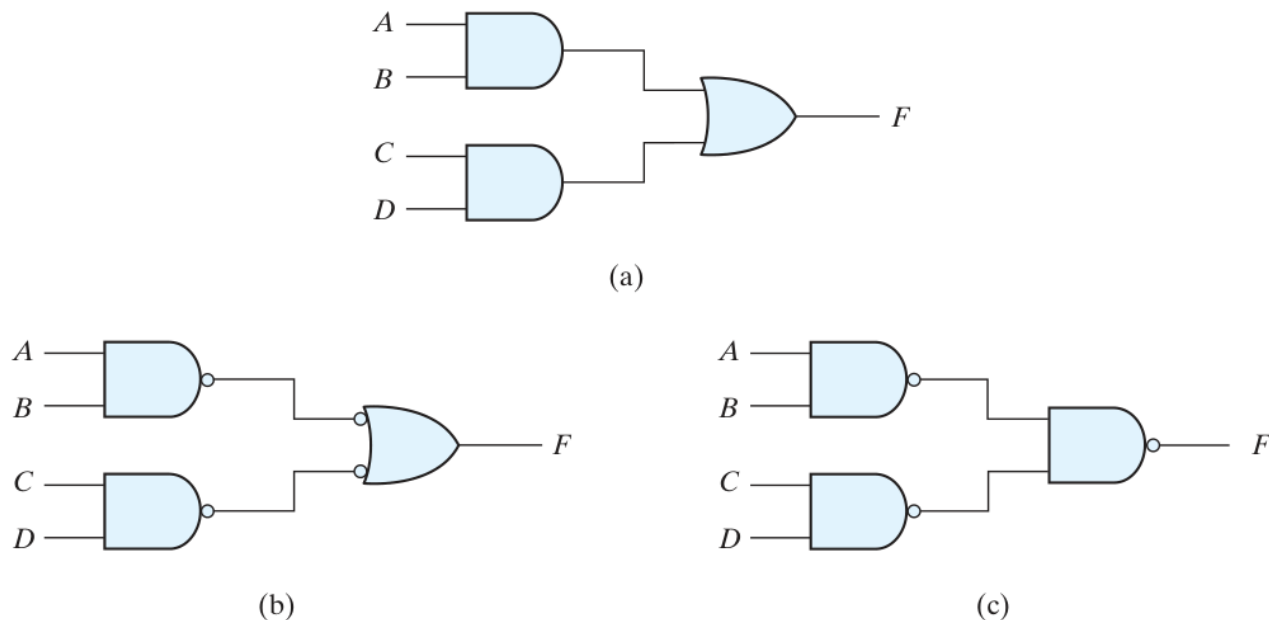


FIGURE 3.18
Three ways to implement $F = AB + CD$

Figure

3.18: Demonstrating equivalence of AND-OR logic and two-level NAND-NAND logic for the function $F = AB + CD$. Sourced from M. Morris Mano & Michael D. Ciletti, *Digital Design* (6th Global Ed.), p. 120 (printed p. 119).

1.7 Exclusive-OR (XOR) and Exclusive-NOR (XNOR) Gates

Source: Kumar (Ch. 4, pp. 173–176); Mano (Ch. 3, pp. 132–137)

[Pacing: Conceptually Important — The Core of Arithmetic & Parity]

Need (Problem-First)

Standard OR includes the case where both inputs are **1** ("inclusive OR"). But in basic arithmetic addition (**1 + 1 = 0 with carry 1**), the sum bit must be **0** when both inputs are **1**. Furthermore, in error-detection (parity checking), we need a circuit that detects whether an odd or even number of bits are asserted.

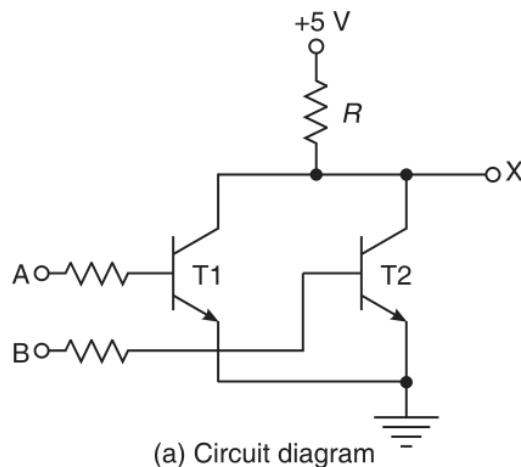
How It Works (The Mechanisms)

1. The XOR Gate (Kumar, p. 173)

- **Plain Words:** The output is HIGH (**1**) if **either input is HIGH, but not both**. The output is **1** if the inputs are different (**01** or **10**), and **0** if they are identical (**00** or **11**).
- **Boolean Equation:**

$$X = A \oplus B = AB + \bar{A}\bar{B}$$

- **Authentic Diagram:**



Input		Output
A	B	X
0 V	0 V	5 V
0 V	5 V	0 V
5 V	0 V	0 V
5 V	5 V	0 V

(a) Circuit diagram

(b) Truth table

Figure 4.19 Discrete two-input NOR gate.

4.6 THE EXCLUSIVE-OR (X-OR) GATE

An X-OR gate is a two input, one output logic circuit, whose output assumes a logic 1 state when one and only one of its two inputs assumes a logic 1 state. Under the conditions when both inputs assume the logic 0 state, or when both the inputs assume the logic 1 state, the output assumes a logic 0 state.

Since an X-OR gate produces an output 1 only when the inputs are not equal, it is called *anti-coincidence* gate or *inequality detector*. The output of an X-OR gate is the modulo sum of two inputs. The name Exclusive-OR is derived from the fact that its output is a 1, only when exclusively one of its inputs is a 1 (it excludes the condition when both the inputs are 1).

The logic symbol and truth table of a two-input X-OR gate are shown in Figures 4.20a and 4.20b, respectively. If the input variables are represented by A and B and the output variable by the expression for the output of this gate is written as $X = A \oplus B = A\bar{B} + \bar{A}B$ and read as 'X equal to A ex-or B'.

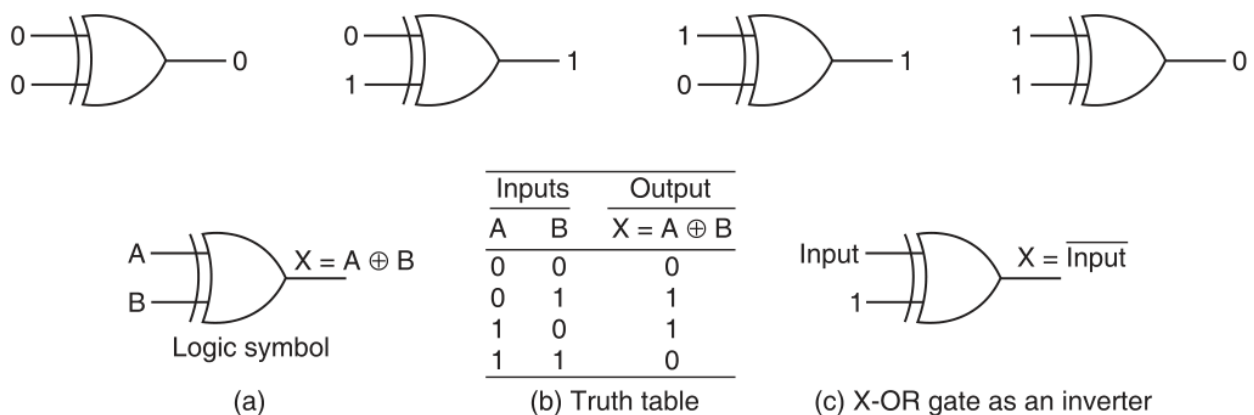


Figure 4.20 Exclusive-OR gate.

Figure

4.20: Exclusive-OR (XOR) gate showing logic symbol, truth table, and gate realization. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 173 (printed p. 141).

2. The XNOR Gate (Equivalence Gate) (Kumar, p. 175)

- **Plain Words:** The exact inverse of XOR. The output is HIGH (1) if both inputs are identical (00 or 11).
- **Boolean Equation:**

$$X = A \odot B = A \oplus B = AB + A \bar{B}$$

• **Authentic Diagram:**

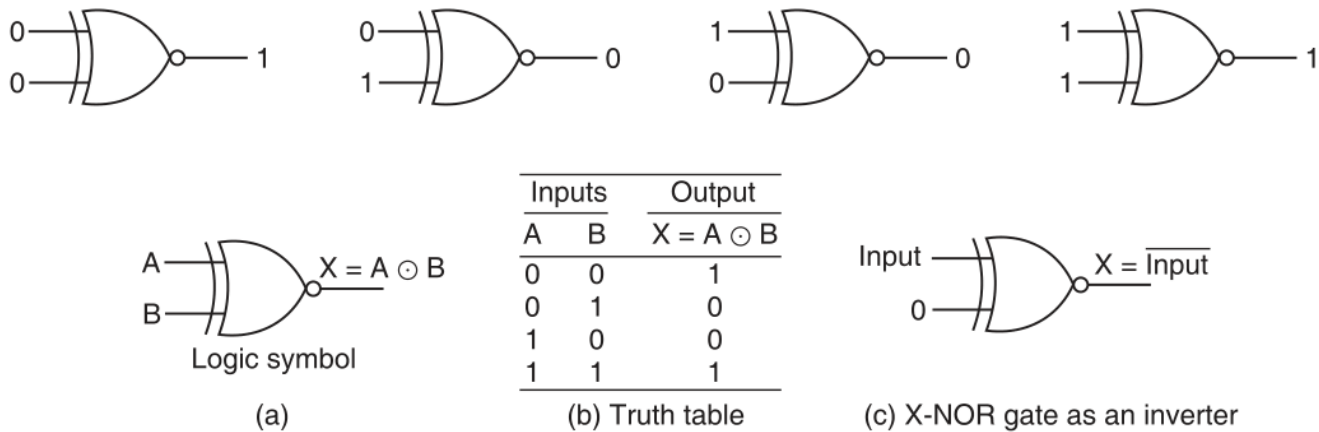


Figure 4.21 Exclusive-NOR gate.

Figure

4.21: Exclusive-NOR (XNOR) gate showing logic symbol and truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 175 (printed p. 143).

Understanding Checkpoint

- **Question:** If $A = 1$ and $B = 1$, what are the outputs of an XOR gate and an XNOR gate?
- **Answer:** XOR output = 0; XNOR output = 1.

PART 3: Digital Logic Families (RTL, TTL, CMOS) and Interfacing

1.8 Logic Family Performance Metrics (Figures of Merit)

Source: Kumar (Ch. 16, pp. 892–897); Mano (Ch. 2, pp. 91–93)

[Pacing: Conceptually Essential — Know What Makes a Family Good or Bad]

Need (Problem-First)

Logic gates are not ideal abstract equations; they are made of physical transistors. When selecting chips for an engineering project, how do you decide whether to use TTL, CMOS, or ECL? We need rigorous quantitative figures of merit to measure their speed, power, driving capability, and electrical robustness.

The Core Figures of Merit Defined Inline:

1. **Propagation Delay (t_{pd}):** The time delay between the application of an input signal transition and the resulting output transition (Kumar, p. 892):
2. **t_{pHL} :** Propagation delay time from HIGH to LOW output.
3. **t_{pLH} :** Propagation delay time from LOW to HIGH output.
4. Average propagation delay: $t_{pd} = \frac{t_{pHL} + t_{pLH}}{2}$. Measured in **nanoseconds (ns)**. Shorter delay means a faster processor.
5. **Power Dissipation (P_D):** The electrical power consumed by the gate, measured in **milliwatts (mW)**:

$$P_D = V_{CC} \times I_{CC}$$

6. **Speed-Power Product (SPP):** The ultimate figure of merit balancing speed against energy consumption (Kumar, p. 894):

$$SPP = t_{pd} \times P_D$$

Measured in **picojoules (pJ)**. A lower SPP indicates a superior, more energy-efficient silicon architecture. 4. **Fan-Out:** The maximum number of standard logic inputs of the same family that the output of a gate can reliably drive without causing output voltages to degrade outside legitimate logic bands (Kumar, p. 893). 5. **Fan-In:** The number of independent input terminals

available on a single gate. 6. **DC Noise Margins (NM_H , NM_L)**: The maximum voltage amplitude of electrical noise that can be tolerated without causing false switching (Kumar, p. 896, Figure 16.4):

where environment noise may be high.

Figure 16.4a shows the range of output voltages that can occur in a logic circuit. Voltages greater than $V_{OH}(min)$ are considered as a logic 1 and voltages lower than $V_{OL}(max)$ are considered as a logic 0. Voltages in the disallowed range should not appear at a logic circuit output under normal conditions. Figure 16.4b shows the input voltage requirements of a logic circuit. The logic circuit will respond to any input greater than $V_{IH}(min)$ as a logic 1 and to any input lower than $V_{IL}(max)$ as a logic 0. Voltages in the indeterminate range will produce an unpredictable response and should not be used.

Figure

16.4: Graphical definition of DC noise margins showing transmitting output voltage levels (V_{OH} , V_{OL}) and receiving input threshold levels (V_{IH} , V_{IL}). Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 896 (printed p. 864).

1.9 Resistor-Transistor Logic (RTL)

Source: Kumar (Ch. 4, p. 173, Figure 4.19; Ch. 16, p. 892)

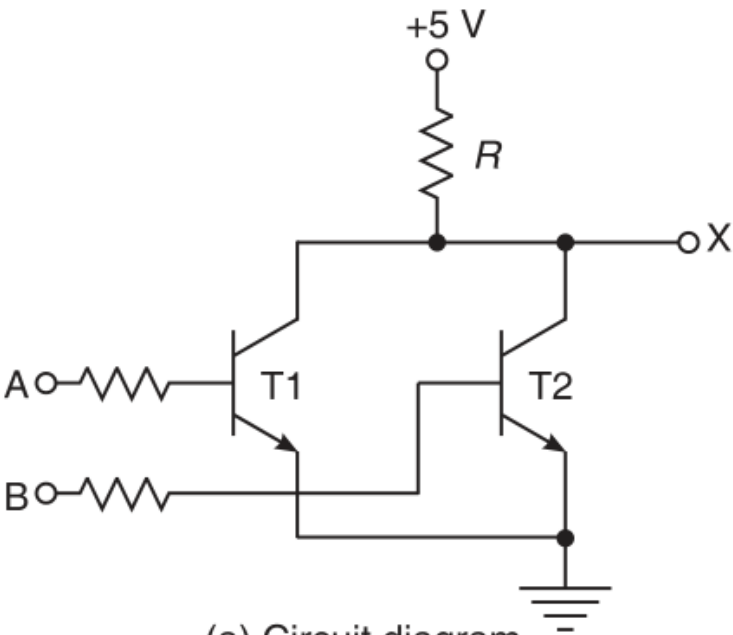
[Pacing: Historical Foundation — How Transistor Logic Began]

Need (Problem-First)

The very first integrated circuits in the 1960s (including the Apollo Guidance Computer) needed the simplest possible way to implement logic using basic bipolar transistors and resistors.

How It Works (The Physical Mechanism)

Figure 4.19 from Kumar (p. 173) shows a discrete two-input RTL NOR gate:



(a) Circuit diagram

Input		Output
A	B	X
0 V	0 V	5 V
0 V	5 V	0 V
5 V	0 V	0 V
5 V	5 V	0 V

(b) Truth table

Figure 4.19 Discrete two-input NOR gate.

Figure

4.19: Discrete two-input Resistor-Transistor Logic (RTL) NOR gate. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 173 (printed p. 141).

Characteristic 13 Walkthrough: Hardware Reality Check

- **Case 1 ($A = 0\text{ V}$, $B = 0\text{ V}$)**: Neither transistor **T_1** nor **T_2** receives base current. Both transistors are in **cutoff (OPEN)**. No current flows through collector resistor **R**. With zero voltage drop across **R**, the output node **X** is pulled directly up to **+5 V**

(Logic 1).

- **Case 2 ($A = 5\text{ V}$ or $B = 5\text{ V}$):** Base current floods the corresponding transistor, driving it into **saturation (CLOSED)**. The saturated transistor creates a low-resistance path from output node **X** directly to Ground (0 V). The output voltage collapses to $V_{CE(sat)} \approx 0.2\text{ V}$ (Logic 0).
- This produces the exact truth table of a **NOR gate**.
- **Fatal Flaw of RTL:** The collector resistor **R** severely limits speed. When switching HIGH, parasitic capacitances must charge through **R**, giving poor rise times, tiny noise margins (0.2 V), and poor fan-out (only 4 to 5 loads).

1.10 Transistor-Transistor Logic (TTL): Totem-Pole, Open-Collector, and Tri-State

Source: Kumar (Ch. 16, pp. 898–907, Figures 16.5, 16.7, 16.8, 16.9)

[Pacing: Conceptually Heavy — Study the Transistor Conduction States Carefully]

Need (Problem-First)

RTL was too slow and noise-sensitive. Engineers realized that if they replaced the passive pull-up resistor with an **active transistor switch** (push-pull / totem-pole), the circuit could actively drive both HIGH and LOW transitions with blazing speed, giving rise to TTL (the 7400 series).

The Standard Totem-Pole TTL NAND Gate (Kumar, p. 898, Figure 16.5)

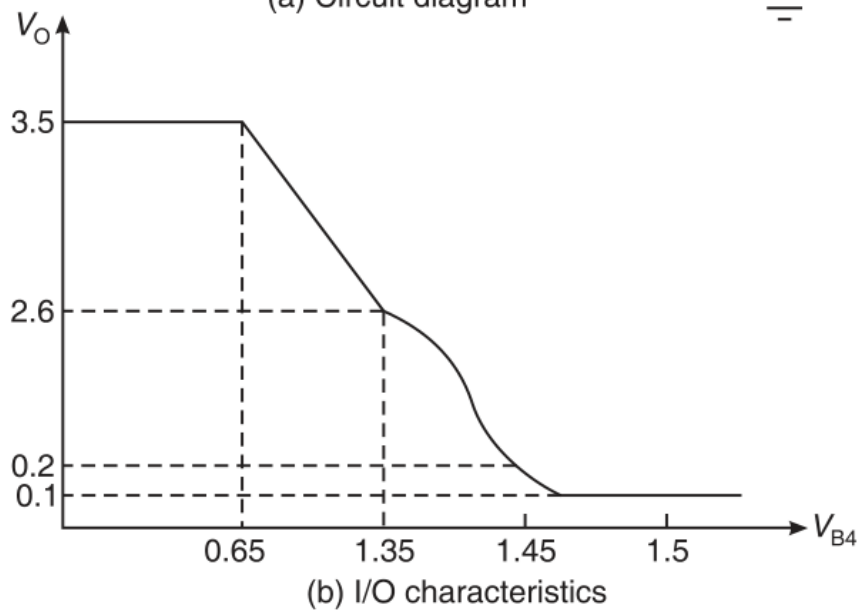
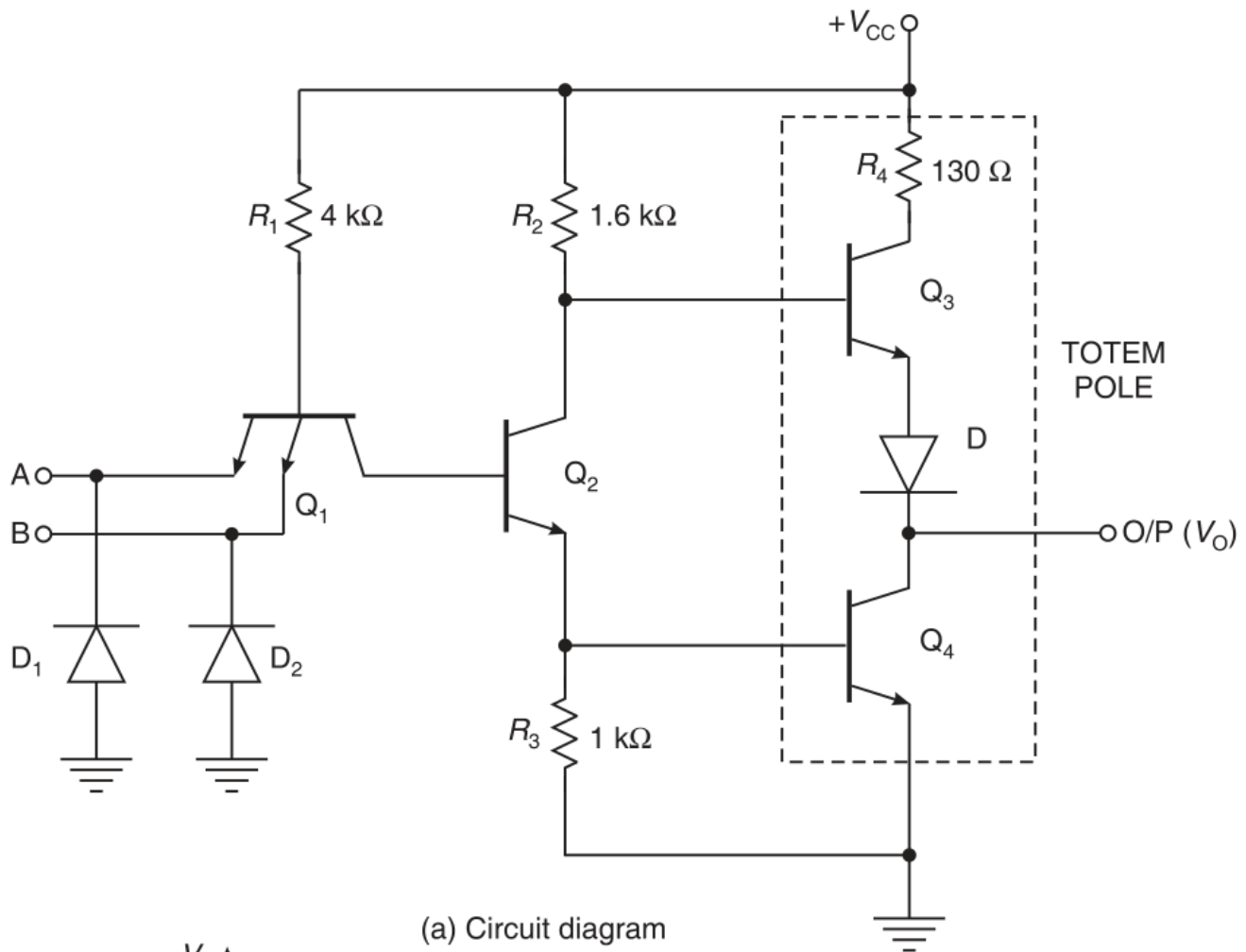


Figure 16.5 TTL NAND gate.

Figure

16.5: Schematic diagram and voltage transfer characteristics of a standard Transistor-Transistor Logic (TTL) NAND gate with active totem-pole output stage. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 898 (printed p. 866).

Four Key Functional Stages in TTL:

1. **Multi-Emitter Input Transistor (Q₁):** Has multiple emitters (one for each input **A**, **B**). It performs the AND operation directly on electrons.

2. **Phase Splitter Transistor (Q₂):** Produces two out-of-phase output voltages simultaneously: its collector voltage and emitter voltage are complementary.
3. **Totem-Pole Output Stage (Q₃, D, Q₄):**
4. **Q₄ (Pull-down transistor):** When turned ON, it pulls the output node down to Ground (0 V).
5. **Q₃ (Pull-up transistor):** When turned ON, it pulls the output node up to +V_{CC} (+5 V).
6. **Diode D:** Crucial engineering component! It ensures Q₃ and Q₄ can **never be ON at the same time**. Without diode D, the base-emitter drops would allow both transistors to conduct simultaneously during transitions, creating a massive current spike from V_{CC} straight to ground.

Characteristic 13 Walkthrough: Hardware Reality Check

- **Condition 1 (Any Input A or B is LOW, 0.2 V):**
Current flows from V_{CC} through R₁ (4 kΩ) and out through the LOW input emitter to ground. The base of Q₁ is clamped to 0.2 V + 0.7 V = 0.9 V. Because Q₂ and Q₄ need 0.7 V + 0.7 V = 1.4 V across their base-emitter junctions to turn ON, Q₂ and Q₄ are **completely OFF (cutoff)**. With Q₂ OFF, no current flows through R₂, pulling the base of Q₃ HIGH. Q₃ turns ON, actively driving the output node V_O to:

$$V_{OH} = V_{CC} - V_{BE3} - V_D = 5.0 \text{ V} - 0.7 \text{ V} - 0.7 \text{ V} \approx 3.6 \text{ V (Logic 1)}$$

- **Condition 2 (Both Inputs A and B are HIGH, 3.5 V):**
The emitter junctions of Q₁ are reverse-biased. Current from V_{CC} through R₁ now flows forward through the base-collector junction of Q₁ directly into the base of Q₂. Q₂ **turns ON hard**, which in turn floods the base of Q₄ with current, turning Q₄ **ON into saturation**. Saturated Q₄ pulls the output node down to ground:

$$V_{OL} = V_{CE4(sat)} \approx 0.2 \text{ V (Logic 0)}$$

Meanwhile, the collector of Q₂ drops to $V_{CE2(sat)} + V_{BE4} \approx 0.2 + 0.7 = 0.9 \text{ V}$. To turn Q₃ ON would require $V_O + V_D + V_{BE3} \approx 0.2 + 0.7 + 0.7 = 1.6 \text{ V}$. Since 0.9 V < 1.6 V, Q₃ is **held completely OFF**. - Output = 0 only when all inputs are 1. **This is a NAND gate!**

Open-Collector TTL Gates & Wired-AND

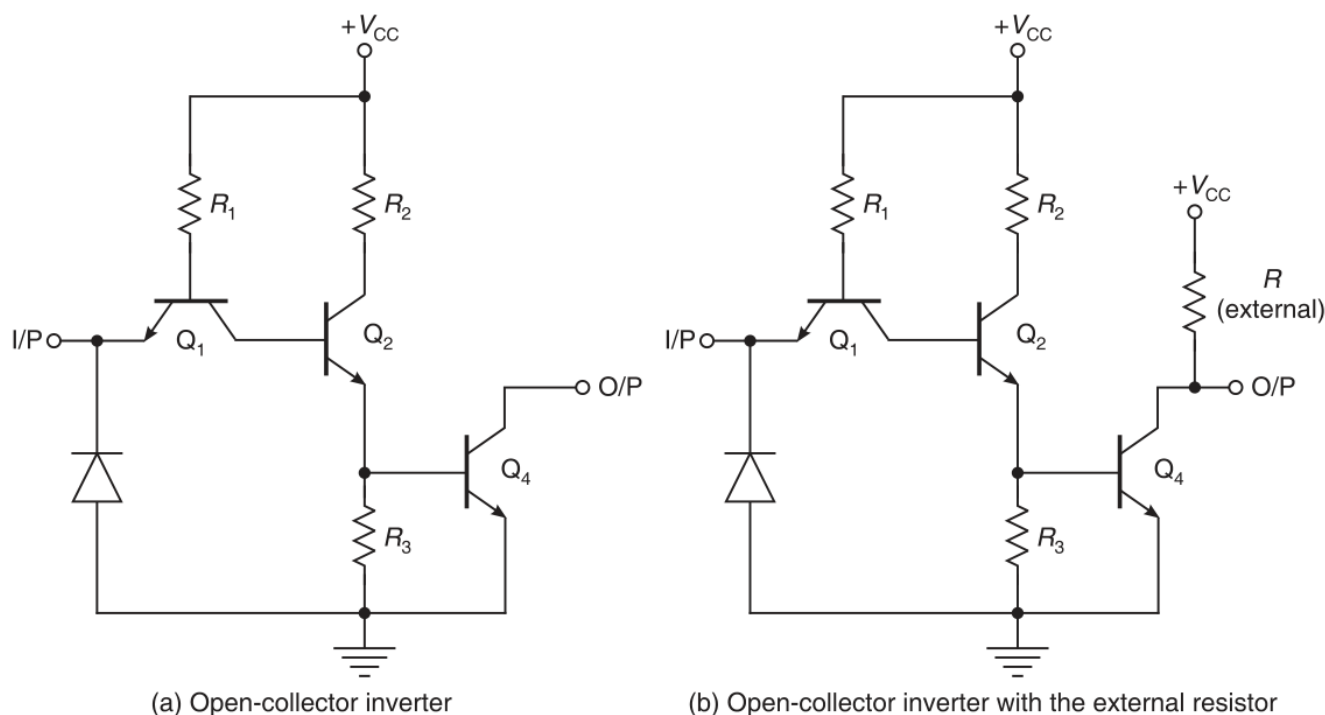
Source: Kumar (Ch. 16, pp. 903–905, Figures 16.7 & 16.8)

Need (Problem-First)

Totem-pole outputs can **never be directly connected together**! If Gate 1 outputs HIGH (+3.6 V) while Gate 2 outputs LOW (0.2 V), a direct low-resistance short circuit is created between V_{CC} and Ground through Q₃ of Gate 1 and Q₄ of Gate 2. Enormous current (>100 mA) will surge through the transistors, permanently destroying both chips. How can multiple gates safely share a single common communication wire (a bus)?

The Solution: Open-Collector Output (Kumar, p. 904, Figure 16.7)

In an **open-collector** gate, the upper totem-pole components (Q₃, diode D, and resistor R₄) are completely omitted. The collector of pull-down transistor Q₄ is left floating in open air.



(a) Open-collector inverter

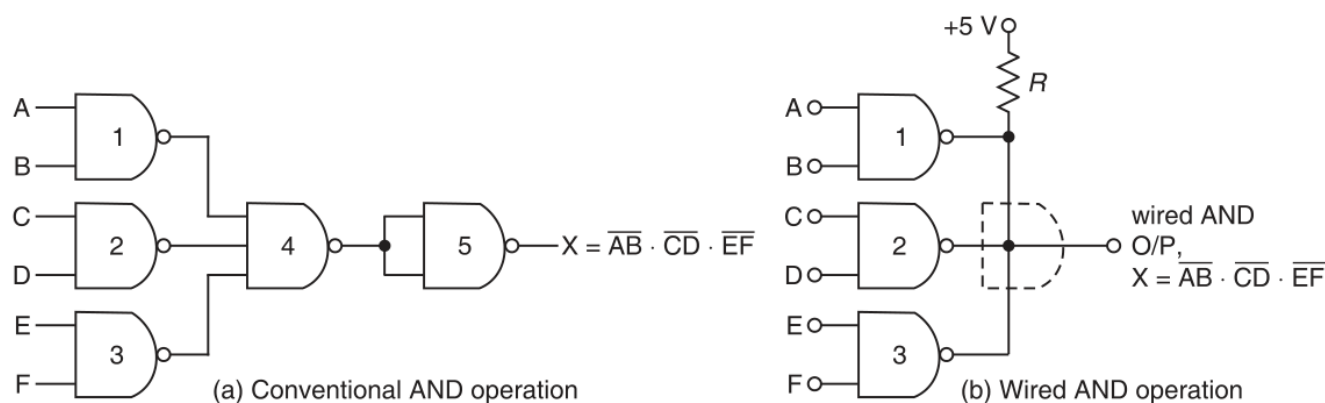
(b) Open-collector inverter with the external resistor

Figure 16.7 Circuit diagram and logic symbol of open-collector inverter.

Figure

16.7: Circuit diagram and logic symbol (showing internal open-collector marking) of an open-collector TTL inverter. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 904 (printed p. 872).

- **Requirement:** An external **pull-up resistor (R_L)** must be connected between the output wire and **$+V_{CC}$** .
- **Wired-AND Operation (Kumar, p. 905, Figure 16.8):** Multiple open-collector outputs can be tied directly to the same wire! If any gate turns its **Q₄** ON, it pulls the shared wire to **0 V**. The wire only reaches **+5 V** if **all** gates release their transistors into cutoff. This performs an automatic **Wired-AND** function without requiring an extra physical AND gate:



(a) Conventional AND operation

(b) Wired AND operation

Figure 16.8 ANDing of TTL gates.

Figure

16.8: Direct wired-AND connection of two open-collector TTL gates using an external pull-up resistor. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 905 (printed p. 873).

Tri-State (Three-State) TTL Logic

Source: Kumar (Ch. 16, pp. 905–907, Figure 16.9)

Need (Problem-First)

Open-collector gates solve the bus contention problem, but they are relatively slow because the output line rises slowly through external resistor **R_L** . We want the blazing speed of an active totem-pole output, but the bus-sharing ability of an open collector.

The Solution: Tri-State Logic

A **tri-state gate** has three distinct output states: 1. **Logic 0**: Low impedance path to Ground (**Q₄** ON, **Q₃** OFF). 2. **Logic 1**: Low impedance path to **+V_{CC}** (**Q₃** ON, **Q₄** OFF). 3. **High-Impedance State (Z / Hi-Z)**: Both **Q₃** and **Q₄** are turned **OFF simultaneously!** The gate physically disconnects itself from the output wire, behaving like an open switch with infinite resistance (**>10 M Ω**).

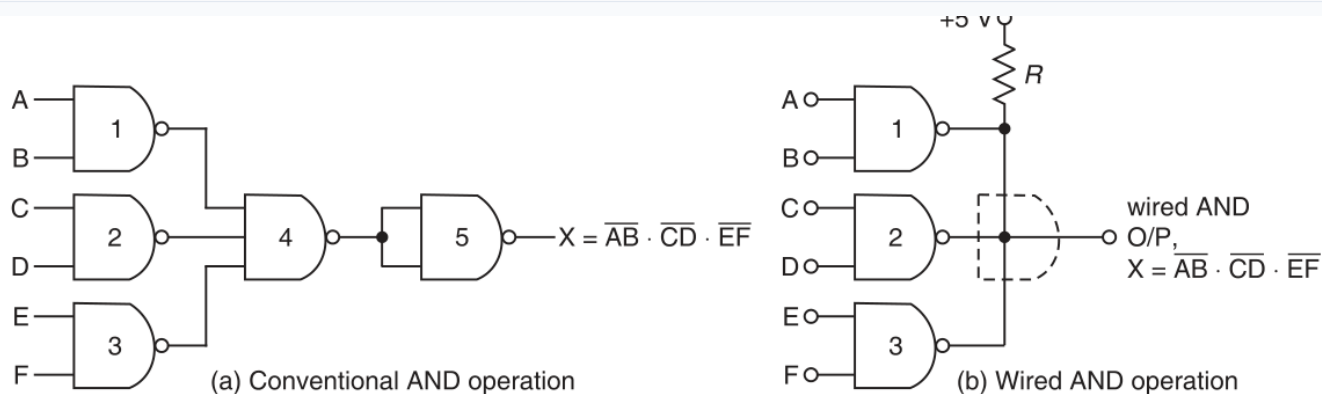


Figure 16.8 ANDing of TTL gates.

Because of the absence of pull-up transistors, the wired-AND connections significantly reduce switching speeds. However, they are useful in reducing the chip count of a system when speed is not a consideration.

16.5.2 Tri-state (3-state) TTL

The third TTL configuration is the tri-state configuration. It utilizes the advantage of high speed of operation of the totem-pole configuration and wire ANDing of the open-collector configuration. It is called the *tri-state* TTL, because it allows three possible output states: HIGH, LOW, and HIGH impedance (Hi-Z). In the Hi-Z state, both the transistors in the totem-pole arrangement are turned off, so that the output terminal is a HIGH impedance to ground or V_{CC} . In fact, the output is an open or floating terminal, that is, neither a LOW nor a HIGH. In practice, the output terminal is not an exact open circuit, but has a resistance of several $M\Omega$ or more relative to ground and V_{CC} .

The circuit of a tri-state inverter is shown in Figure 16.9a. The tri-state operation is obtained by modifying the basic totem-pole circuit of Figure 16.5. The circuit has two inputs—A is the normal logic input and E is an enable input that can produce the Hi-Z. The traditional symbol is shown in Figure 16.9b.

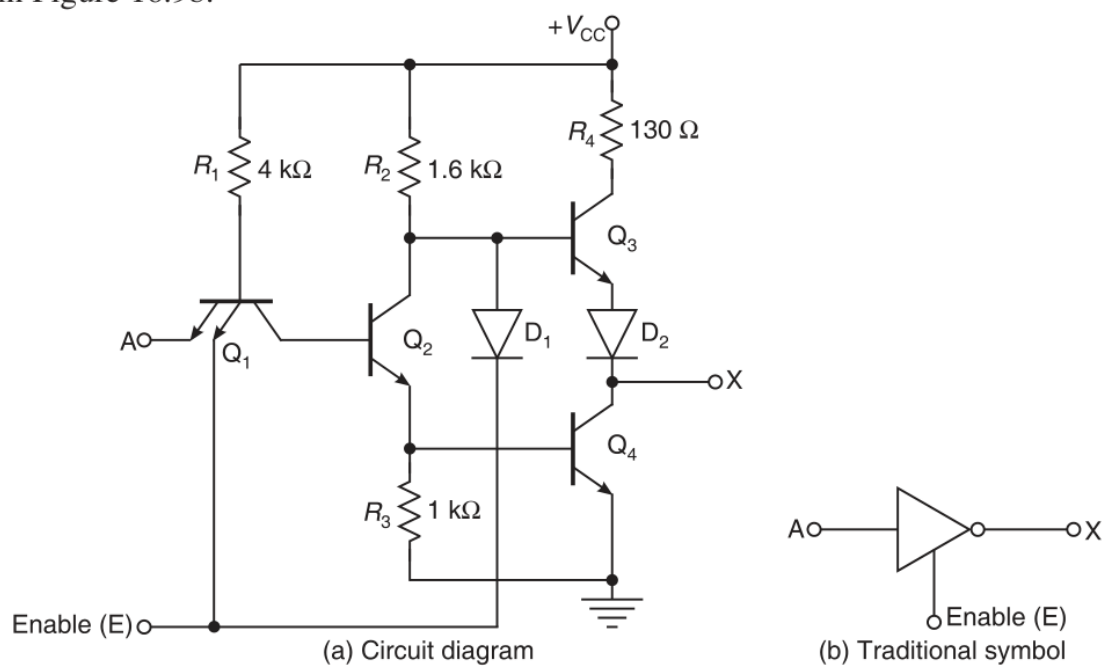


Figure 16.9 Tri-state TTL inverter.

Figure

- **Operation:** When the **Enable** line is asserted, the gate behaves as a normal high-speed inverter. When Enable is de-asserted, internal diode steering holds the bases of both **Q_3** and **Q_4** at ground, placing the output into the floating **Z** state. This enables microprocessors to connect hundreds of memory and peripheral chips to a single shared data bus.

1.11 CMOS Logic: Inverter, NAND, and NOR Gates

Source: Kumar (Ch. 16, pp. 917–924, Figures 16.23, 16.24, 16.25)

[Pacing: The King of Modern VLSI — Master This Section Thoroughly]

Need (Problem-First)

TTL gates consume substantial continuous electric power because current flows through internal resistors even when the gate is not switching. In a modern computer processor with billions of transistors, TTL logic would consume kilowatts of power and instantly melt the silicon chip. The world needed a logic family that draws **virtually zero electric power** when idling. That technology is **CMOS** (Complementary Metal-Oxide-Semiconductor).

The Secret of CMOS: Complementary Pairs

CMOS pairs an **n-channel MOSFET (NMOS)** with a **p-channel MOSFET (PMOS)**: - **NMOS (Q_1)**: Turns **ON** when Gate voltage is HIGH (**+5 V**), and turns **OFF** when Gate is LOW (**0 V**). Connected between output and Ground (the **Pull-Down Network**). - **PMOS (Q_2)**: Turns **ON** when Gate voltage is LOW (**0 V**), and turns **OFF** when Gate is HIGH (**+5 V**). Connected between **+V_{DD}** and output (the **Pull-Up Network**).

1. The CMOS Inverter (Kumar, p. 921, Figure 16.23)

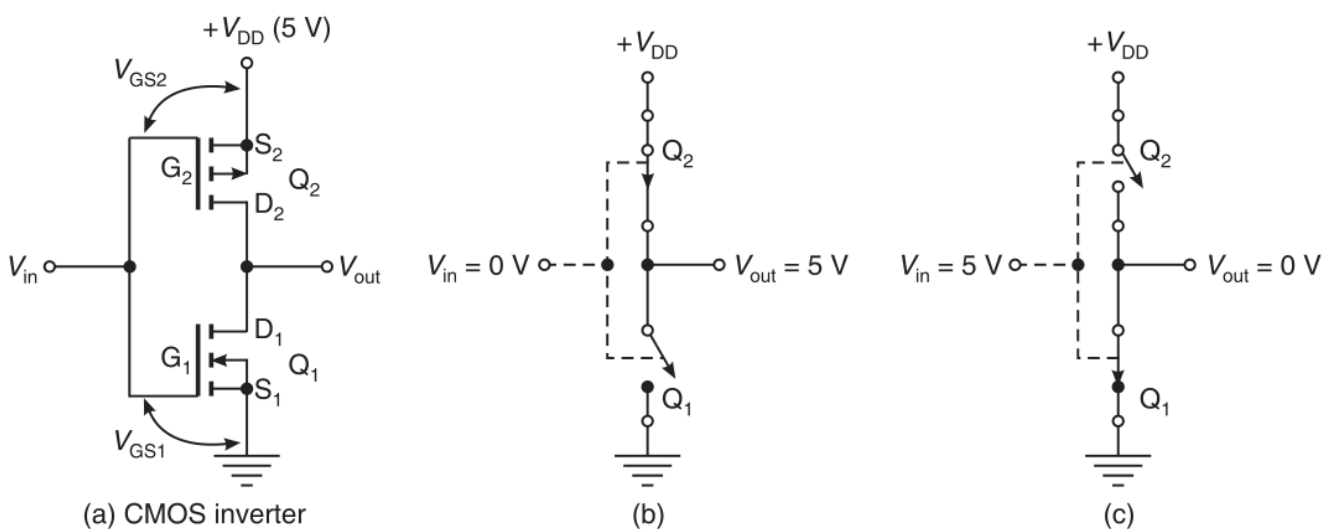


Figure 16.23 Circuit diagram and equivalent circuits for various inputs of the CMOS inverter.

Figure

16.23: CMOS Inverter schematic and equivalent switch circuits for LOW and HIGH input states. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 921 (printed p. 889).

Characteristic 13 Walkthrough: Hardware Reality Check

- When $V_{in} = 0 \text{ V}$ (LOW):
 $V_{GS1} = 0 \text{ V}$ ■ NMOS **Q_1** is **OFF** (OPEN switch).
 $V_{GS2} = -5 \text{ V}$ ■ PMOS **Q_2** is **ON** (CLOSED switch).
 Output node V_{out} is connected directly to $+V_{DD}$ through the conducting PMOS channel, while completely isolated from Ground.

$$V_{out} = +V_{DD} = +5 \text{ V (Logic 1)}$$

- When $V_{in} = +5 \text{ V}$ (HIGH):
 $V_{GS1} = +5 \text{ V}$ ■ NMOS **Q_1** is **ON** (CLOSED switch).

$V_{GS2} = 0 \text{ V}$ ■ PMOS Q_2 is **OFF (OPEN switch)**.

Output node V_{out} is connected directly to Ground through the conducting NMOS channel, while completely isolated from $+V_{DD}$.

$$V_{out} = 0 \text{ V (Logic 0)}$$

- **Why Static Power is ZERO:** In both steady-state conditions, **one of the two series transistors is always completely OFF**. There is never a direct DC path from $+V_{DD}$ to Ground! Current only flows for a few picoseconds during the actual transition while charging the load capacitance.

2. The CMOS NAND Gate (*Kumar*, p. 922, Figure 16.24)

- **Architecture:** Two PMOS transistors (Q_1 , Q_2) connected in **parallel** between $+V_{DD}$ and output; two NMOS transistors (Q_3 , Q_4) connected in **series** between output and Ground.

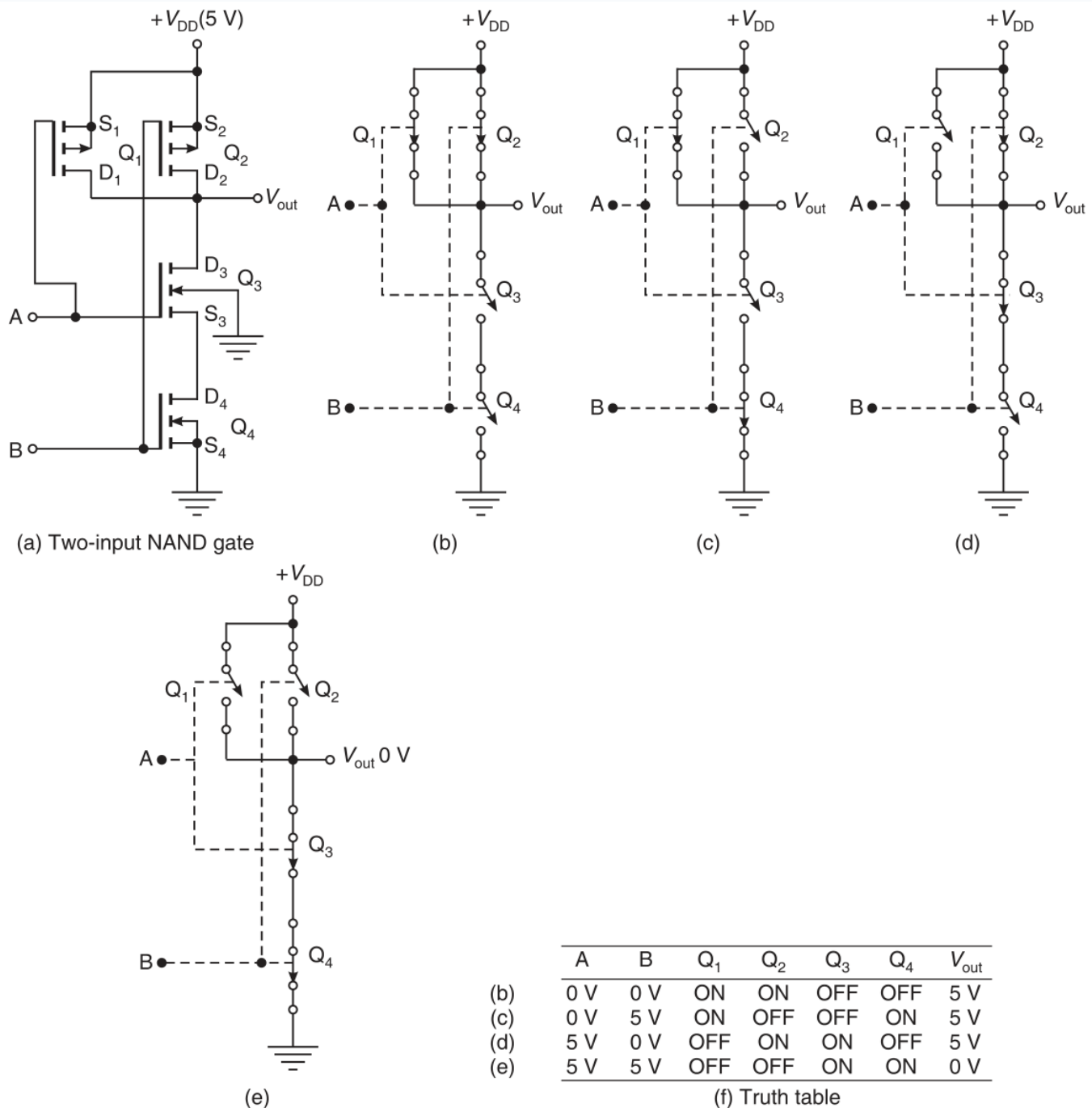


Figure 16.24 Circuit diagram and equivalent circuits for various inputs of the CMOS NAND gate.

Figure

16.24: CMOS two-input NAND gate schematic, equivalent switch circuits, and operational truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 922 (printed p. 890).

- **Operation:**
- If *either* A or B is 0 V, the corresponding PMOS is ON, pulling V_{out} to +5 V. Because NMOS are in series, the path to ground is broken.
- Only when *both* A and B are +5 V are both series NMOS (Q₃, Q₄) ON simultaneously, while both parallel PMOS are OFF. Output is pulled to 0 V.

3. The CMOS NOR Gate (Kumar, p. 923, Figure 16.25)

- **Architecture:** Two PMOS transistors (Q₁, Q₂) connected in **series** between +V_{DD} and output; two NMOS transistors (Q₃, Q₄) connected in **parallel** between output and Ground.

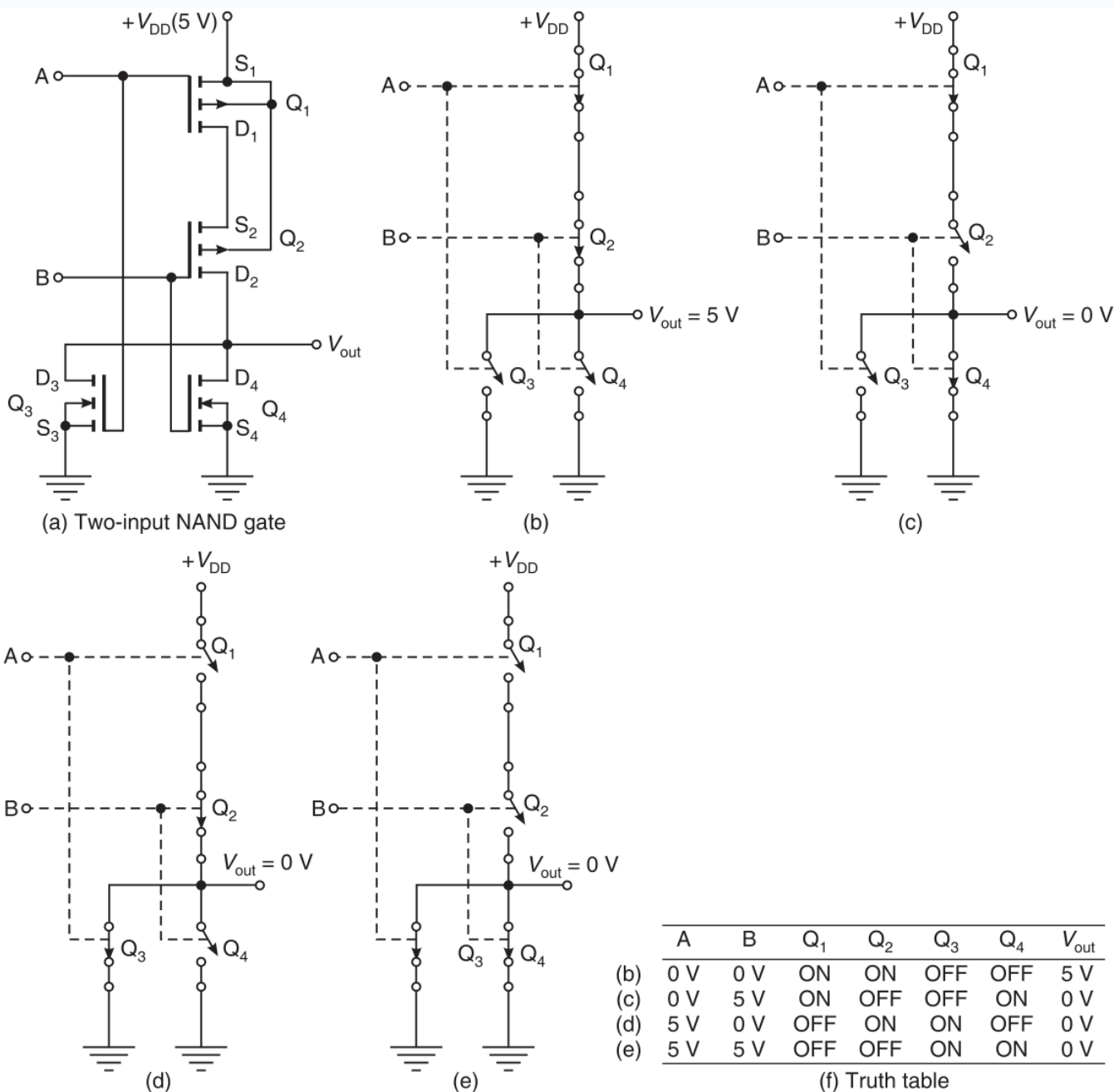


Figure 16.25 Circuit diagram and equivalent circuits for various inputs of the CMOS NOR gate.

Figure

16.25: CMOS two-input NOR gate schematic, equivalent switch circuits, and operational truth table. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits* (4th Ed.), p. 923 (printed p. 891). (Note: Sub-label (a) contains an authentic textbook typographical misprint labeling the schematic as NAND; the circuit is verified as NOR by its series PMOS and parallel NMOS configuration and verified caption).

- **Operation:**
- If either **A** or **B** is **+5 V**, the corresponding parallel NMOS conducts, clamping **V_{out}** to **0 V**.
- Only when both **A** and **B** are **0 V** are both series PMOS transistors ON simultaneously, pulling **V_{out}** to **+5 V**.

1.12 Interfacing CMOS and TTL Logic Families

Source: Kumar (Ch. 16, pp. 930–933, Figures 16.32 & 16.33)

[Pacing: Critical Practical Engineering — Read Deliberately]

Need (Problem-First)

In practical engineering systems, you often need to connect a TTL microprocessor to a modern CMOS memory chip, or vice versa. If you blindly connect a copper wire between their pins without analyzing voltage and current compatibility, the circuit will fail to recognize logic states or destroy the chips.

The Fundamental Voltage & Current Mismatch

Parameter	Standard TTL (5 V)	Standard CMOS (5 V)	Compatibility Conflict
$V_{OH(min)}$ (Output High Min)	+2.4 V to +2.7 V	+4.9 V	TTL output high is too low for CMOS!
$V_{IH(min)}$ (Input High Min)	+2.0 V	+3.5 V (70% of V_{DD})	CMOS requires ≥ 3.5 V to see a 1
$V_{OL(max)}$ (Output Low Max)	+0.4 V	+0.1 V	Compatible (0.4 V < 1.5 V)
$V_{IL(max)}$ (Input Low Max)	+0.8 V	+1.5 V (30% of V_{DD})	Compatible

Case 1: TTL Driving CMOS (Kumar, p. 931, Figure 16.32)

- The Physical Problem:** Standard TTL guarantees a minimum output HIGH voltage of only $V_{OH} = 2.4$ V to 2.7 V. But a 5 V CMOS gate requires at least $V_{IH} = 3.5$ V to reliably recognize a HIGH! A standard TTL output falls directly into the CMOS forbidden/undefined region.
- The Solution:** Connect an external pull-up resistor (R_p , typically 1 k Ω to 3.3 k Ω) from the interconnect wire to $+V_{CC}$ (Kumar, Figure 16.32a), or use a dedicated TTL-compatible CMOS buffer (such as the 74HCT series). When the TTL output goes HIGH, R_p pulls the line all the way up to $+5.0$ V, well above the CMOS 3.5 V threshold.

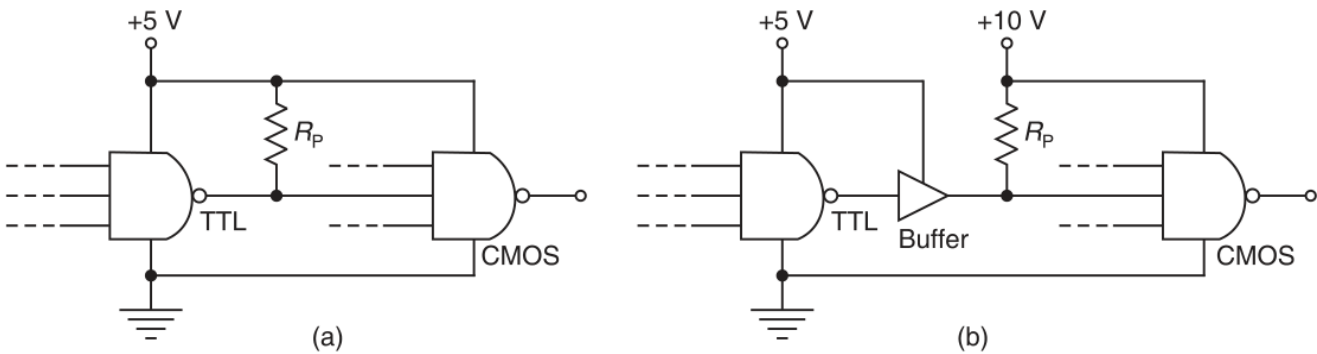


Figure 16.32 TTL to CMOS interfacing.

Figure

16.32: Interfacing TTL to CMOS logic using an external pull-up resistor or a supply-level shifting transistor. Sourced from A. Anand Kumar, Fundamentals of Digital Circuits (4th Ed.), p. 931 (printed p. 899).

Case 2: CMOS Driving TTL (Kumar, p. 932, Figure 16.33)

- Voltage Compatibility:** CMOS $V_{OH(min)} = 4.9$ V, which easily exceeds TTL $V_{IH(min)} = 2.0$ V. CMOS $V_{OL(max)} = 0.1$ V, which is well below TTL $V_{IL(max)} = 0.8$ V. Voltages are 100% compatible!
- The Current Sinking Problem:** When a TTL input is driven LOW, current physically flows out of the TTL emitter into the driving CMOS output ($I_{IL} \approx 1.6$ mA per standard TTL gate). A standard 4000-series CMOS gate can only sink about 0.4 mA to 1.0 mA before its internal NMOS channel resistance causes V_{OL} to rise above 0.8 V!
- The Solution:** Use a high-current CMOS buffer (such as 74HCT / 74ACT) or limit the fan-out to a single low-power Schottky TTL (74LS) input:

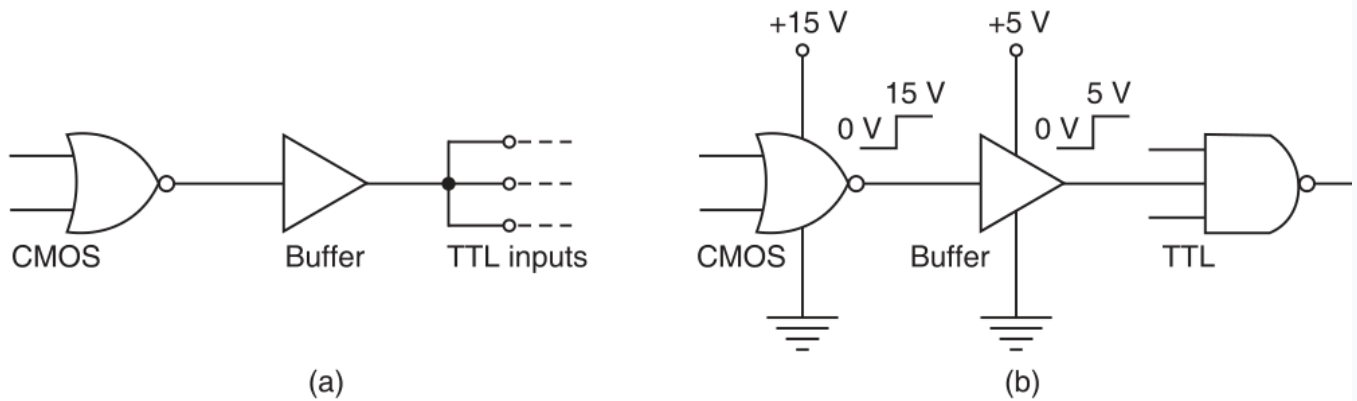


Figure 16.33 CMOS to TTL interfacing.

Figure

16.33: Interfacing CMOS to TTL showing buffer requirements for driving standard TTL loads. Sourced from A. Anand Kumar, *Fundamentals of Digital Circuits (4th Ed.)*, p. 932 (printed p. 900).

Unit I Understanding Checkpoints & Misconception Audit

1. Does This Make Sense? Self-Check

- **Question 1:** Why is two-level NAND-NAND logic completely equivalent to AND-OR logic?
- **Answer:** By De Morgan's theorem, an inverted-input OR gate (bubble-OR) is logically identical to a NAND gate ($A + B = \overline{\overline{A} \cdot \overline{B}}$). The output bubbles of the first-stage NAND gates cancel the input bubbles of the second-stage NAND gate, preserving the original AND-OR function.
- **Question 2:** Why must you never directly tie two standard totem-pole TTL outputs together?
- **Answer:** If one outputs HIGH and the other outputs LOW, a direct, destructive short circuit is created from V_{CC} to Ground through their low-impedance transistors, burning out the devices.
- **Question 3:** What component is mandatory when interfacing a standard TTL output to a standard CMOS input?
- **Answer:** A pull-up resistor to $+V_{CC}$, because standard TTL output HIGH ($\approx 2.7 \text{ V}$) is below the CMOS input HIGH threshold (3.5 V).

2. Conceptual Misconceptions Corrected

- **Misconception:** "A CMOS gate consumes zero power."
Correction: It consumes virtually zero *static* (idle) power. But every time the output switches between 0 and 1, internal capacitances must be charged and discharged ($P_{\text{dynamic}} = C \cdot V^2 \cdot f$). At high clock frequencies (gigahertz), CMOS chips consume immense power and generate significant heat!
- **Misconception:** "In 2's complement subtraction, if an end-carry is produced, you must add it back to the LSB."
Correction: No! That is the obsolete rule for 1's complement. In 2's complement, an end carry past the MSB is **strictly discarded**.